

LangGraph Cheatsheet

ToolNode, checkpoint, streaming, human-in-the-loop, subgraph & store.

langgraph.prebuilt · langgraph.checkpoint · langgraph-turkce.github.io

1. ToolNode & Hazır Ajanlar

ToolNode + tools_condition

```
from langgraph.prebuilt import ToolNode, tools_condition

g.add_node("tools", ToolNode(tools))
g.add_conditional_edges("chatbot", tools_condition)
g.add_edge("tools", "chatbot")
```

create_react_agent

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(llm, tools=[arac1])
agent.invoke({"messages": [{"user", "...}]})
```

Standart görevler için hazır; özel akışlarda elle graf kur.

2. Functional API

@task & @entrypoint

```
from langgraph.func import entrypoint, task

@task
def ozetle(metin: str) -> str: ...

@entrypoint(checkpointer=MemorySaver())
def akis(girdi: dict) -> str:
    return ozetle(girdi["metnin"]).result()
```

Graph API'ye alternatif; aynı checkpoint/interrupt altyapısını kullanır.

Graph API mi, Functional API mi?

- Doğrusal, az dallanma → Functional API
- Karmaşık dallanma / çoklu-ajan → Graph API

İkisi bir arada da kullanılabilir.

3. Checkpointer & Kalıcılık

MemorySaver (sadece dev)

```
from langgraph.checkpoint.memory import MemorySaver

app = g.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "u-42"}}
app.invoke({"messages": [...]}, config)
```

Production: Postgres / Redis

```
from langgraph.checkpoint.postgres import PostgresSaver
# yüksek trafik: langgraph.checkpoint.redis.RedisSaver

with PostgresSaver.from_conn_string(DB_URI) as cp:
    cp.setup()
    app = g.compile(checkpointer=cp)
```

MemorySaver süreç yeniden başladığında veriyi kaybeder. Redis, çok yüksek trafikte düşük gecikme sağlar.

4. Streaming

stream_mode karşılaştırması

- **values** — her adımdan sonra tam state
- **updates** — sadece değişen kısım
- **messages** — LLM token akışı (chat UI)

```
for c in app.stream(input, config, stream_mode="updates"):
    print(c)
```

astream_events

Retriever, tool, LLM token gibi tüm olayları ayrı yakalar — UI'da canlı durum göstermek için.

```
async for ev in app.astream_events(input, config, version="v2"):
    if ev["event"]=="on_chat_model_stream": ...
```

5. Human-in-the-loop

interrupt_before

```
app = g.compile(checkpointer=cp,
    interrupt_before=["tools"])

app.invoke(input, config) # tools'tan önce durur
app.invoke(None, config) # onaydan sonra devam
```

State'i güncelleyip devam etme

```
app.update_state(config, {"tutar": 400})
app.invoke(None, config)
```

Devam etmeden önce state manuel değiştirilebilir.

6. Subgraph & Store

Subgraph — modülerlik

```
alt_graph = alt_builder.compile()
ana_graph.add_node("alt_akis", alt_graph)
```

Store — thread'ler arası hafıza

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
store.put(("u123",), "tercih", {"dil": "tr"})
```

Checkpointter tek thread'e özeldir; Store thread'ler arası kalıcıdır.

7. Config Injection & Test Yazma

RunnableConfig ile veri geçirme

```
def chatbot(state: State, config: RunnableConfig):
    kid = config["configurable"].get("kullanici_id")
    ...
```

State'e ait olmayan (kullanıcı kimliği, model seçimi) çağrıya özel veriler için kullanılır.

Sahte (fake) modelle test

```
from langchain_core.language_models.fake_chat_models import
FakeListChatModel

sahte_llm = FakeListChatModel(responses=["cevap"])
```

Gerçek API'ye ödeme yapmadan graf mantığını deterministik test eder.

8. Hızlı Referans

KAVRAM	NE İŞE YARAR
ToolNode	Hazır araç çalıştırıcı node
tools_condition	Hazır araç-çağırma karar fonksiyonu
MemorySaver / PostgresSaver	Checkpointter backend'i
thread_id	Konuşma/oturum kimliği
InMemoryStore / PostgresStore	Uzun süreli hafıza (embedding destekli anlamsal arama dahil)
interrupt_before/after	Human-in-the-loop durma noktası
NodeInterrupt	Koşullu, veri-bazlı durma istisnası
RunnableConfig	Çalışma zamanı parametre enjeksiyonu

Sık yapılan hata: Production'da MemorySaver kullanmak — süreç yeniden başladığında tüm konuşma geçmişi kaybolur.